

The Alberta Buck - Publication and Deployment (DRAFT v0.1)

Kernels to crates.io, platform and contracts to npm and PyPI, and
the road to a public repository

Perry Kundert

2026-07-25 12:00:00

The BUCK codebase has one consumer today: this checkout. Every entry point is a `make` target, every import path is a `sys.path` shim or a relative `../..` into a sibling directory, and every binary artifact is gitignored because it is rebuilt locally on demand.

The **Alberta Buck game** – a board projected on a TV, personal devices showing private net worth, real economic decisions played out against real BUCK mechanics – is the forcing function that ends that. A game runs on someone else's machine. That requires published, versioned, installable artifacts across three registries, and it requires them in an order the dependency graph dictates rather than the order of our enthusiasm.

Two facts govern everything below. First: **publishing is publishing**. A private GitHub repository and a public PyPI project are entirely compatible – but the sdist we upload contains the source, so whatever we publish is public whether or not the repository is. The question is never "can we publish while private" (we can); it is "which code are we ready to make public". Second: **the first upload is irreversible**. crates.io versions cannot be deleted, PyPI filenames cannot be reused, npm unpublish is a 72-hour window. Names and version numbers are burned on first use.

Contents

1	The short answer	4
2	Decisions required before the first upload	4
2.1	D1: the public-code boundary	4
2.2	D2: licence – CAL-1.0 for the kernels	5
2.2.1	What we are actually trying to accomplish	5
2.2.2	The layers	6
2.2.3	Consequences to plan for	7
2.2.4	Deliverables	7
2.3	D3: names	8
2.4	D4: version policy and tag scheme	9
2.5	D5: how the compiled kernels reach PyPI	10
2.6	D6: where the game’s authority lives	11
3	The artifact matrix	11
4	Phase P0: pre-flight, common to every registry	12
5	Phase P1: crates.io – the four kernel crates	14
6	Phase P2: npm – the kernel and the JS platform	15
7	Phase P2.5: alberta-buck-contracts – the compiled contracts	16
7.1	How the contracts are consumed today	16
7.2	What ships	17
7.3	Reproducibility: pin the compiler	18
7.4	Prototype status	18
7.5	Steps	18
8	Phase P3: PyPI – alberta-buck-core (pure Python)	19
9	Phase P4: PyPI – alberta-buck-kernel (binary wheels)	20
10	Phase P5: PyPI – alberta-buck	20
11	Phase P6: what the game adds	21
12	Phase P7: making the repository public	22
13	Risks and open questions	23

14 Status board

24

1 The short answer

Two orderings, depending on which goal comes first.

Fast path to pip install alberta-buck (2 uploads). `alberta_buck` hard-imports `buck_core` (`sim/chain.py`, `sim/mini.py`), but only the *pure-Python* part of it – `session.py`, `artifacts.py`. The compiled kernels are optional, gated behind `BUCK_IDENTITY_BACKEND` (see `alberta_buck/wallet/_kernel.py`). So the minimum viable PyPI presence is:

```
alberta-buck-core (pure python wheel, py3-none-any) -> alberta-buck
```

No Rust toolchain in CI, no wheel matrix, no cross-compilation. Days, not weeks. Two caveats: the `alberta_buck` sim – including the equilibrium research – becomes public source on PyPI (D1); and an installed `alberta-buck` **cannot deploy a world** until the contract artifacts ship too (P2.5), because `load_artifact` reads `out/` from a repo checkout.

Full path. The kernels first, because the Holochain zones the game wants are Rust crates that will depend on `alberta-buck-identity` and `alberta-buck-wallet` by version, not by path; and because the browser needs the wasm build and the contract artifacts on npm. This is the recommended order if the game is the priority rather than the Python package:

```
crates.io: alberta-buck-math -> -identity -> -registry -> -wallet
           npm: alberta-buck-kernel -> alberta-buck-contracts -> alberta-buck-core
           PyPI: alberta-buck-core -> alberta-buck-contracts
                  -> alberta-buck-kernel (binary wheels)
                  -> alberta-buck
```

Everything in both paths is achievable with the repository private. Nothing in either path requires making it public; going public (Phase P7) is a separate decision with its own checklist.

2 Decisions required before the first upload

These are irreversible-adjacent. Each has a recommendation; none is settled until you say so.

2.1 D1: the public-code boundary

Publishing a Python package publishes its source. A PyPI sdist is a tarball of the package directory; a wheel is a zip of `.py` files. There is no "binary

only" option for Python that means anything. npm tarballs are source. Only the Rust crates ship as compiled artifacts downstream – and **cargo publish** uploads the crate source too.

So the boundary must be drawn per artifact, not per repository:

artifact	contents	public today?
alberta-buck-math et al. (crates)	the monetary/identity kernels	yes – these are the
alberta-buck-kernel (npm, PyPI)	the same kernels, as wasm / as .so	yes
alberta-buck-core (npm, PyPI)	session, journal, agents, artifact loader	yes – infrastructure
alberta-buck-contracts	compiled ABI + bytecode, prototype	yes, <i>clearly marked</i>
alberta-buck (PyPI)	the sim, the equilibrium work, the quotes	your call

Recommendation: publish the kernels, both platform libraries and the contract artifacts now; hold **alberta-buck** until you are content that the sim and the equilibrium findings (**EQUILIBRIUM.md**, the sweep vectors, **rebalance_policy.py**) are ready to be read by anyone. The papers are the natural gate: publish the package when the paper it accompanies is out.

2.2 D2: licence – CAL-1.0 for the kernels

Rule (settled 2026-07-25): start restrictive and relax deliberately.

A licence can always be loosened later, or granted separately to a specific counterparty; it can never be tightened on a version already published. Every ambiguity therefore resolves toward the more restrictive choice.

The repository state this replaces – inconsistent, and with **no LICENSE file at all**:

where	says
src/*.sol (49 files)	GPL-3.0 / GPL-3.0-or-later
pyproject.toml	GPL-3.0-or-later
core/rust/*/Cargo.toml (all 4)	MIT
core/js/package.json	MIT
core/python/pyproject.toml	(nothing)
repository root	no LICENSE, no COPYING

2.2.1 What we are actually trying to accomplish

The goal is that an organization intending to use the Alberta Buck to build a *liberty-restricting* system – custodial control of other people’s keys, freezeable or seizable balances, users denied their own records – is at minimum operating out of licence.

No OSI-approved licence can do this by restricting *purpose*: OSD 6 forbids discrimination against fields of endeavour, and the ethical-licence family (Hippocratic and relatives) is closer to a statement of values than a working legal instrument.

The **Cryptographic Autonomy License** (CAL-1.0, OSI-approved 2020, drafted for Holochain) does it by restricting *architecture* instead, which is both enforceable and self-executing. Its distinguishing obligations:

section	obligation
4.1	provide Recipients the Source Code (or unrestricted network access to it)
4.1.2	modifications go out under CAL-1.0 or a compatible licence
4.1.3	90-day embargo permitted for coordinated vulnerability disclosure
4.2.1	give each Recipient a no-charge copy of their own User Data
4.2.2	no cryptographic or technological measure that limits a Recipient's access to function
4.2.3	no contractual restriction on a Recipient exercising the same permissions independently
4.5	Combined Work Exception : per-file opt-in permitting combination into a larger work un

A system that holds users' keys, can freeze their balances, or withholds their records **cannot comply with 4.2 while using CAL-covered code**. It is out of licence the day it ships – no intent needs to be proven, and no field of endeavour is discriminated against. This is the mechanism we want, and it is what CAL was commissioned to do.

2.2.2 The layers

- **Kernels** (`alberta-buck-{math,identity,wallet,registry}`), **the wasm/PyO3 bindings, and both platform libraries: CAL-1.0**. This is where the leverage is. Anyone building a control system needs the identity, wallet and Notes kernels; nobody needs our sim. CAL anywhere else is decorative.
- **Contracts and their artifacts: GPL-3.0-or-later** – already true, and not a change: all 49 `src/*.sol` files declare it today. See P2.5 for the third-party artifacts (Uniswap) that must not be redistributed under it.
- `alberta_buck` (`sim`, `research`, `tooling`): GPL-3.0-or-later, as `pyproject.toml` already claims. It is research; it is not deployed at users, so CAL's user-data machinery has nothing to bite on.
- **Papers and documents**: separate decision, not blocking any upload.

2.2.3 Consequences to plan for

- **The Combined Work Exception is the partner path, and it is one-way.** Marking a file CWE cannot be undone for a published version. So mark it narrowly and late – at the *binding and API surface* (`bindings/py`, `bindings/js`, the `core/js` public entry points) if and when a real integrator needs it – never across the kernels wholesale. A credit union can then link the kernel into a proprietary back office and still owe its members their keys and their data, which is exactly the trade we want.
- **Dual licensing is the enforcement lever**, and it works only because you hold the copyright. An organization that cannot comply must come and ask, and you may decline. This requires a **CLA** the moment the repository is public and outside contributions arrive (P7) – without it, contributed code is not yours to relicense.
- **Expect friction, and budget for it.** CAL is young, has thin case law, and every institutional counsel will have to read it rather than recognize it. That friction is the instrument working, not a defect – but it is a real cost against Phase 6 adoption.
- **The licence governs code, not ideas.** The papers are public; a determined actor can reimplement the mechanism from them. What CAL buys is that they cannot do it *with our code* while denying their users autonomy.
- **Why not Apache-2.0** (the runner-up, and the choice if CAL ever proves unworkable): its substantive advantage over MIT is the express patent grant with retaliation-termination, plus an explicit trademark non-grant – both of which matter to a bank’s legal review. MIT is silent on patents. CAL, drafted by a patent attorney, carries its own patent provisions; confirm their scope with counsel before relying on them.
- **No dependency conflict:** our Rust stack (`arkworks`, `pyo3`, `wasm-bindgen`, `serde`, `sha2`) is uniformly MIT/Apache, so copyleft on top is clean.

2.2.4 Deliverables

- **LICENSE** at the root (GPL-3.0), **core/LICENSE** (CAL-1.0), and a **NOTICE** recording third-party licences (OpenZeppelin MIT; Uniswap v3-core

BUSL-1.1 -> GPL-2.0-or-later since its 2023-04-01 change date; v3-periphery GPL-2.0; universal-router GPL-3.0).

- SPDX identifiers in every manifest and file header: `CAL-1.0`, `CAL-1.0-Combined-Work-Exception`, `GPL-3.0-or-later` – all valid SPDX, so cargo, npm and PyPI meta-data accept them unchanged.
- `license` / `license-files` keys in all nine manifests; the four crates move from MIT to `CAL-1.0`.

2.3 D3: names

Rule (settled 2026-07-25): every published distribution is prefixed `alberta-buck-`, on every registry. The same name on two registries names that language’s build of the same thing – `alberta-buck-core` is the Python platform on PyPI and the JS platform on npm, deliberately.

Checked 2026-07-25; every name below is unregistered on its registry:

registry	distributions	status
crates.io	<code>alberta-buck-math</code> , <code>-identity</code> , <code>-registry</code> , <code>-wallet</code>	available
npm	<code>alberta-buck-kernel</code> , <code>-core</code> , <code>-contracts</code> , <code>-game</code>	available
PyPI	<code>alberta-buck</code> , <code>alberta-buck-core</code> , <code>-kernel</code> , <code>-contracts</code>	available

Three consequences worth stating so they are not mistaken for oversights:

- **Import names do not change.** `import buck_core`, use `buck_math::`, `import { ... } from "alberta-buck-core"`. Distribution name and import name are different things, and your own `python-slip39` (which imports `slip39`) already sets the precedent. For the crates this means an explicit `[lib] name = "buck_math"`, since cargo otherwise derives `alberta_buck_math` from the package name.
- **Directory names do not change.** `core/rust/buck-math` stays; cargo does not require the directory to match the package. Renaming directories would churn the Makefile, `.cargo/config.toml`, every test path and every doc reference for no gain.
- **Reserve the `@alberta-buck` npm scope anyway** (free, one command). We publish unscoped prefixed names per the rule above, but an unclaimed scope matching a published prefix is an invitation to typosquatting.

2.4 D4: version policy and tag scheme

Three registries, one repository. A single `v0.1.0` tag that fires all three release workflows breaks the moment one artifact is unchanged (crates.io rejects a republished version).

Recommendation: prefixed tags, one workflow each.

tag	fires	publishes
<code>rs-v*</code>	<code>release-crates.yml</code>	the four kernel crates, in dependency order
<code>js-v*</code>	<code>release-npm.yml</code>	<code>alberta-buck-kernel</code> , <code>-contracts</code> , <code>-core</code>
<code>py-v*</code>	<code>release-pypi.yml</code>	<code>alberta-buck-core</code> , <code>-contracts</code> , <code>-kernel</code> , <code>alberta-buck</code>

Lockstep at major.minor; patch floats (settled 2026-07-26). The kernels are one artifact compiled three ways, so the bit-identical guarantee has to be anchored to a version – but it is a claim about *semantics*, and only major.minor can change semantics. Therefore:

- `alberta-buck-math 0.1.*` in Rust, JS and Python compute identically. The cross-language vector suites are the enforcement, and they may assume it.
- A patch release in one language does **not** oblige the others. Packaging fixes are the common case and they are language-local: the JS packages went to 0.1.1 for a `wasm-opt` size pass that did not touch a line of Rust, and forcing four identical crates to 0.1.1 to match would have been ceremony, not information.
- A major or minor bump is a cross-language event. It lands everywhere, or the guarantee above stops being true.

Consequences worth stating: the inter-crate requirements stay caret ("`0.1`", so `0.1.x` siblings resolve), and `release-crates.yml` guards that they track the workspace major.minor rather than pinning exact patches. Skew *within* a minor is expected and fine; skew *across* one is a bug.

`alberta-buck-contracts` is outside this entirely: it versions with the *contracts*, not the kernels (P2.5). `0.0.1` everywhere was a placeholder; public life starts at **0.1.0**.

Add a CI guard that the tag matches the manifest version (a released `py-v0.1.0` that uploads `0.0.1` is a burned filename). Take the `slip39` pattern for Python: `version = {attr = "alberta_buck.version.__version__"}` so there is one place to bump.

2.5 D5: how the compiled kernels reach PyPI

Today `core/python/buck_core/` contains four `.so` files, three of which are **byte-identical copies of one dylib** (`buck_identity.so`, `buck_wallet.so`, `buck_registry.so`: 1,491,824 bytes each, one `cdylib` with three `#[pymodule]` entry points). That trick is right for the Makefile and wrong for a wheel: it triples 1.5 MB for nothing, and it makes `alberta-buck-core` platform-specific – which would drag the entire Rust wheel matrix onto the critical path of `pip install alberta-buck`.

Recommendation: split the distribution along the seam the runtime already has.

<code>alberta-buck-core</code>	pure Python, py3-none-any wheel. <code>session.py</code> , <code>artifacts.py</code> , <code>journal</code> . Hard dependency of <code>alberta-buck</code> . No Rust in CI.
--------------------------------	---

<code>alberta-buck-kernel</code>	binary, abi3-py311 wheels per platform. Optional: <code>'pip install alberta-buck[kernel]'</code> . Selected at runtime by <code>BUCK_IDENTITY_BACKEND</code> , as today.
----------------------------------	---

Keep `import buck_core.buck_identity` working by making `buck_core/buck_identity.py` a shim that re-exports from `buck_kernel` when installed, so no calling code changes.

Within the kernel wheel, ship the dylib **once**. Python's extension loader takes the module name and the file separately, and looks up `PyInit_<name>` in whatever file it is given:

```
# buck_kernel/buck_wallet.py -- one .so, three modules
import sys, importlib.util
from importlib.machinery import ExtensionFileLoader
from pathlib import Path

_so = str(Path(__file__).with_name("_kernel.abi3.so"))
_spec = importlib.util.spec_from_file_location(
    __name__, _so, loader=ExtensionFileLoader(__name__, _so))
_mod = importlib.util.module_from_spec(_spec)
_spec.loader.exec_module(_mod)
sys.modules[__name__] = _mod
```

4.9 MB of wheel becomes 1.9 MB. Verify on each target platform before committing to it; the fallback (ship the dylib three times) is merely wasteful, not broken.

2.6 D6: where the game’s authority lives

This determines whether the Python side must be published at all for the game to exist, so it belongs here rather than in the game’s own design doc.

The mechanics you describe – savings in BuckBasket, real BuckCredit and BUCK issuance, time passage, a random-walk or narrative-driven basket – can be authoritative in one of three places:

1. **Python server** (`alberta_buck.sim.server` already does this: `pyrevm` worlds at interactive speed, websockets, multiple clients). Highest fidelity: the real contracts, the real EVM. Requires a host, and `pyrevm` has **no linux-aarch64 wheels and no macOS wheels past cp312** (checked 0.3.7) – so the deployment target must be `x86_64`, or build from source with a Rust toolchain. **Needs `alberta-buck-contracts`.**
2. **Browser EVM** (`tevm`, per player or per board). Local-first, no server, already exercised by `core/js/test/*.tevm.test.js` – but `Tevm` is the slow backend, and `farmsim.mjs` already OOMs past ~6 simulated months. **Needs `alberta-buck-contracts`.**
3. **Kernel-only** – the game computes on `alberta-buck-math` / `alberta-buck-wallet` directly, with no EVM anywhere. The kernels exist precisely because they are bit-identical to the contracts; a game does not need EVM semantics, it needs the arithmetic to be right, and periodic vector-checks against forge keep it honest. Runs in the browser at native speed via `wasm`; runs in a Holochain zone unchanged.

Recommendation: **(3), with (1) as the fidelity anchor.** It is the only option that is local-first (which is the whole point of choosing Holochain), the only one that runs on a phone, and it makes the `wasm` kernel – not the Python server – the game’s critical dependency. Under that choice the game needs `npm` and `crates.io`; `PyPI` becomes the research distribution rather than the game’s runtime.

Note that (3) does *not* make the contract artifacts optional: they are how the fidelity anchor is built, they are what the vector generator compiles, and any "show me this on a real chain" moment needs them. Publish them regardless – see P2.5.

3 The artifact matrix

artifact	registry	kind	source	needed by
alberta-buck-math	crates.io	rlib	core/rust/buck-math	everything
alberta-buck-identity	crates.io	rlib	core/rust/buck-identity	wallet, regist
alberta-buck-registry	crates.io	rlib	core/rust/buck-registry	wallet, zome
alberta-buck-wallet	crates.io	rlib	core/rust/buck-wallet	zomes, SDK
alberta-buck-kernel	npm	wasm + js glue	core/rust/bindings/js*	the game, br
alberta-buck-contracts	npm	ABI + bytecode	out/ via bundle-artifacts	any EVM-ba
alberta-buck-core	npm	ESM	core/js	the game, pa
alberta-buck-core	PyPI	pure wheel	core/python	alberta-bu
alberta-buck-contracts	PyPI	pure wheel	out/ via a new bundler	alberta-bu
alberta-buck-kernel	PyPI	abi3 wheels	core/rust/bindings/py*	fast Python
alberta-buck	PyPI	pure wheel	alberta_buck/	research, sim

Not published (for now): the Solidity *sources* (the compiled artifacts suffice for consumers; sources follow when the repository opens), the papers, the equilibrium sweep vectors, *circuits/*.

4 Phase P0: pre-flight, common to every registry

Everything here is a prerequisite for the *first* upload of anything, and none of it depends on which registry goes first.

1. **Licence files** (D2). Root LICENSE (GPL-3.0), core/LICENSE (CAL-1.0), NOTICE for the third-party terms; SPDX identifiers and license keys in all manifests.
2. **Commit core/rust/Cargo.lock**. It is currently gitignored ("lockfile committed once binding crates land" – they have landed). Published wheels and wasm must be reproducible from a tag.
3. **Repository metadata** in every manifest: `repository`, `homepage`, `readme`, `keywords`, `categories` (crates.io), `rust-version`, `classifiers` (PyPI). crates.io rejects publishes missing `description` or `license`; the rest is discoverability.
4. **Vendor the golden vectors into the crates**. Today every crate's test suite reaches outside its own directory:

crate	reads
alberta-buck-math	../../test/vectors/math-vectors.json
alberta-buck-identity	../../test/vectors/identity.json, ../../vectors/identity.json
alberta-buck-registry	../../vectors/registry-kernel-vectors.json
alberta-buck-wallet	../../vectors/wallet-kernel-vectors.json

`cargo package` only includes files under the crate directory, so a published crate would ship a test suite that compiles and then panics on the missing file for every downstream user. For crates whose entire value proposition is "vector-proven bit-identical", that is unacceptable. Copy each needed vector file into the crate (`core/rust/buck-math/tests/vectors/`), add a `make core-vectors-sync` that refreshes them, and a CI byte-compare that fails on drift. (`alberta-buck-identity`'s `build.rs` is fine: it reads `constants/poseidon_constants.json` from inside the crate, and that file is tracked.)

5. **Path dependencies need versions.** `alberta-buck-wallet` depends on `buck-identity` by bare path today; `crates.io` requires a version: `alberta-buck-identity = { path = "../buck-identity", version = "0.1" }`. The path is unchanged (directories do not rename, D3); the dependency *key* becomes the new package name, while `[lib] name = "buck_identity"` keeps use `buck_identity::` working in the source. The existing `kernel = { path = ..., package = "buck-identity" }` aliases in the bindings crates must be repointed at the new package names too.
6. **Secrets and history audit.** Already verified clean: `.env` has never been committed (only `.env.example`), and `.gitignore` covers it. The `.git` directory is 216 MB, dominated by tracked large files (`circuits/ec/powers/bn254*_pows.c` at 3.2/2.7 MB, PDFs, ~1.5 MB equilibrium vectors) – fine for a clone, worth knowing before P7. Re-run the audit immediately before going public, not now.
7. `.github/workflows/`. The repository has none. Create:
 - `ci.yml` – the **release gate**: the three vector suites, which need neither `forge` nor `anvil` (`cargo test`, `pytest core/python/tests`, `node --test core/js/test/*.vectors.test.js`). Full `forge/anvil` CI is a bigger lift and is not on this critical path.
 - `release-crates.yml`, `release-npm.yml`, `release-pypi.yml` (P1-P5).

Release jobs should use plain toolchains (`dtolnay/rust-toolchain`, `actions/setup-node`, `actions/setup-python`) rather than `nix` – matching `crypto-licensing` and `python-slip39`, and avoiding a `nix` bootstrap on every publish. Note that Actions minutes on a **private** repository are metered against the account quota; public repositories are free.

Accept: `make nix-core-test` passes from a clean checkout; every manifest carries licence + repository metadata; the vector-drift check is green.

5 Phase P1: crates.io – the four kernel crates

Why first: the Holochain zomes are Rust, and a zome that depends on `alberta-buck-identity` by git path is not something a third party can build. Also the cheapest phase – no binaries, no matrix, no wasm.

Mechanics worth knowing:

- crates.io **does** support trusted publishing (`rust-lang/crates-io-auth-action@v1`, OIDC, no stored token) – **but the crate must already exist**. So the first publish of each crate is manual, with a token, from your machine; trusted publishing is configured afterwards for every release thereafter. (PyPI is the opposite – P3.)
- Publish order is forced by the dependency graph, and each publish must land before the next begins (crates.io indexes asynchronously; allow a minute): `alberta-buck-math -> -identity -> -registry -> -wallet`.
- `cargo publish --dry-run` and `cargo package --list` before each. Read the file list: it is what the world gets.
- The `bindings/*` crates are build artifacts, not libraries. Do not publish them initially; nothing consumes `alberta-buck-math-py` as a crate.

Steps:

1. P0 items 1-5 for the four crates; rename per D3 (`name`, `[lib] name`, the `package` = aliases in dependents); bump to 0.1.0.
2. `cargo publish -p alberta-buck-math` (token), then `-p alberta-buck-identity`, `-p alberta-buck-registry`, `-p alberta-buck-wallet`.

3. Configure trusted publishing for each crate on crates.io: repository `pjkundert/alberta-buck`, workflow `release-crates.yml`.
4. Add `release-crates.yml` (tag `rs-v*`), with the `auth` action and an ordered `publish`, gated on `cargo test`.

Accept: `cargo add alberta-buck-wallet` in an empty project compiles, and its test suite runs green from the published tarball; a `rs-v0.1.1` tag publishes without a token in the workflow.

6 Phase P2: npm – the kernel and the JS platform

Why second: this is what the game actually runs on.

Two packages (the third, `alberta-buck-contracts`, is P2.5):

- `alberta-buck-kernel` – the `wasm-pack` output. Today the Makefile builds `--target nodejs` into `core/js/wasm/` and `--target web` into `core/js/demo/wasm-web/`. A published package must carry both and select via conditional exports:

```
"exports": {
  ".": {
    "node": "./node/buck_identity.js",
    "browser": "./web/buck_identity.js",
    "default": "./web/buck_identity.js" },
  "./math": {
    "node": "./node/buck_math.js",
    "browser": "./web/buck_math.js" }
}
```

- `alberta-buck-core` – `core/js/src`, depending on `alberta-buck-kernel` by version instead of reaching into `../wasm`.

Gotchas, in rough order of likelihood to bite:

1. **The `wasm` output is gitignored** (`/core/js/wasm/`). npm falls back to `.gitignore` when there is no `.npmignore`, so the published tarball would contain **no `wasm` at all**. Fix with an explicit `"files": ["node", "web", "src"]` allowlist – files always wins.
2. **Enable `wasm-opt`**. Both binding crates set `wasm-opt = false` ("one less moving part"). Right for development; wrong for a package a phone downloads – `buck_identity_bg.wasm` is 1,228,545 bytes unoptimised. Turn it on for release builds and measure.

3. **tevm and viem as dependencies** pulls a very large tree into every consumer. Make them **peerDependencies** (optional) so a browser game that only wants the kernel and the wallet does not install an EVM.
4. **Trusted publishing** requires npm $\geq 11.5.1$, Node $\geq 22.14.0$, and GitHub-hosted runners. Note: npm does **not** generate provenance attestations for packages published from a **private** repository, even when the package is public. If provenance matters, that is an argument for sequencing P7 (public repo) before the npm release – otherwise accept its absence for now and gain it later.
5. **package.json repository** must match the GitHub repo exactly.

Steps: reserve the @alberta-buck scope (D3); restructure `core/js` into the packages; add a `core-build-wasm-dist` target emitting both targets with `wasm-opt` on; first publish manually with 2FA; configure trusted publishers; add `release-npm.yml` (tag `js-v*`).

Accept: a scratch Vite app does `npm i alberta-buck-core`, signs an identity and verifies a receipt in the browser with no build-time wasm plumbing; `node --test` against the published tarball replays the wallet vectors.

7 Phase P2.5: alberta-buck-contracts – the compiled contracts

7.1 How the contracts are consumed today

Worth stating plainly, since the question is live: **nothing is baked in by address**. Every world – Python sim, JS sim, browser demo, test – deploys the contracts fresh into its own EVM and learns the addresses from the deploy receipts. There is no fixed-address deployment anywhere, on any chain, and no deployment registry to publish.

What is consumed is (`abi`, `bytecode`) pairs, by two different mechanisms:

side	mechanism
Python	<code>buck_core.artifacts.load_artifact(name)</code> reads <code>out/<X>.sol/<X>.json</code> after <code>repo_ro</code>
JS	<code>core/js/bin/bundle-artifacts.mjs</code> emits <code>core/js/artifacts/bundle.mjs</code> – 427,910 by

So the JS side **already does exactly what you proposed**: a static, importable bundle of compiled code and ABIs. It is simply generated, git-ignored, and rebuilt locally (`make nix-core-js-artifacts`; a bundle test

hashes it against out/ and fails when stale). Publishing is mostly promotion of an existing artifact to a versioned one.

The Python side has a **blocker hiding in it**: `repo_root()` raises `FileNotFoundError` when no `foundry.toml` is reachable. An `alberta-buck` installed from PyPI has no checkout, so today it can import, but cannot deploy a world. Publishing the artifacts is therefore not optional polish – it is a prerequisite for the Python package to do anything (P5).

7.2 What ships

One logical package, two registries:

```
alberta-buck-contracts/
  contracts.json      {name: {abi, bytecode, deployedBytecode}} for the
                      11 forge-built contracts + UniversalRouter
  compiler.json       solc version, optimizer runs, via_ir, evm_version,
                      commit hash of the sources
  deployments.json    {chainId: {name: address}} -- EMPTY for now; the
                      slot exists so adding a testnet deploy is not a
                      breaking change
  index.mjs / index.d.ts  (npm)   named exports + artifacts(name)
  buck_contracts/__init__.py (PyPI) load(name) -> (abi, bytecode)
```

The contract set is the NEEDED list already in `bundle-artifacts.mjs`: `MockERC20`, `UniswapV3Factory`, `UniswapV3Pool`, `SimLP`, `IdentityRegistry`, `BuckCredit`, `BuckKControllerDirect`, `Buck`, `WETH9`, `BuckBasketProRata`, `BuckBasketUniswapV3`, plus the vendored `UniversalRouter` (not forge-built – flag its distinct provenance in the package README).

Ship only the contracts we own. Four of those entries are other people’s copyleft code, and publishing their compiled artifacts is redistribution under their terms, not ours:

dependency	licence in lib/
<code>openzeppelin</code>	MIT (fine, attribution only)
<code>v3-core</code>	BUSL-1.1, Change Date 2023-04-01 -> now GPL-2.0-or-later
<code>v3-periphery</code>	GPL-2.0
<code>universal-router</code>	GPL-3.0

The clean fix costs nothing: consume Uniswap’s **own** published artifact packages (`@uniswap/v3-core`, `@uniswap/v3-periphery`, `@uniswap/universal-router`)

as dependencies of `alberta-buck-contracts`, and ship only our own GPL-3.0-or-later contracts in it. A `NOTICE` file records the rest. Bundling everything into one tarball would make us the redistributor of four licences we do not control, for no benefit to anyone.

7.3 Reproducibility: pin the compiler

Published bytecode that cannot be reproduced is a liability, and the current settings are deliberately loose: `foundry.toml` has `auto_detect_solc = true` while the Makefile forces `--use 0.8.28` (sidestepping a 0.8.31 IR codegen bug), and the Uniswap V3 sources build under a separate profile with their own solc. For the published artifact:

- add a `[profile.release]` (or `dist`) in `foundry.toml` pinning `solc_version`, `optimizer_runs`, `via_ir`, `evm_version = "cancun"`;
- record those settings in `compiler.json` alongside the git commit;
- build the published bundle **only** from that profile, in CI, never from a developer's `out/`;
- keep `deployedBytecode` so a future Etherscan verification is possible.

7.4 Prototype status

These are unaudited prototype contracts for a monetary system. The package README and the PyPI/npm descriptions must say so in the first line, and the version should stay `0.x` until that changes. A `0.1.0` that someone deploys to mainnet because it was on npm is the failure mode to design against.

7.5 Steps

1. Add `make contracts-dist`: build with the pinned profile, emit `dist/contracts/` (the JSON files above), hash-stamp it.
2. Generalise `bundle-artifacts.mjs` to write both the npm package and the Python package data from that one directory.
3. Add the fallback in `buck_core.artifacts.load_artifact`: try `out/` via `repo_root()` first (developer path, unchanged), then the installed `buck_contracts` bundle. A checkout keeps behaving exactly as it does now; an installed wheel starts working.

4. Publish to npm (`js-v*`) and PyPI (`py-v*`); make `alberta-buck` depend on `alberta-buck-contracts`.
5. Optional, and cheap once the bundle exists: drop `contracts.json` into the deck/demo static assets on `perry.kundert.ca`, so a browser demo can `fetch()` it without an npm install.

Accept: `pip install alberta-buck alberta-buck-contracts` in a clean venv, outside any checkout, deploys a BUCK world on pyrevm; `npm i alberta-buck-contracts` + Tevm does the same in a browser tab; the artifact hash is identical between the npm tarball, the wheel and the CI build.

8 Phase P3: PyPI – alberta-buck-core (pure Python)

The short pole for `pip install alberta-buck`, and mercifully simple: no Rust, one universal wheel.

Mechanics: PyPI's **pending publishers** let a project be created by its first OIDC-authenticated upload – so unlike crates.io, this can be zero-token from the very first release. Configure at `pypi.org/manage/account/publishing/`: project name `alberta-buck-core`, owner `pjkundert`, repository `alberta-buck`, workflow `release-pypi.yml`, environment `pypi`. (A pending publisher does not reserve the name; it is claimed on first use.) Private repositories are expected to work – the OIDC claim set has no visibility field – but verify on the first release rather than assuming.

Steps:

1. Split `buck_core` per D5: pure Python here, `.so` files out.
2. Fill in the manifest: name `alberta-buck-core`, licence, description, README, repository, classifiers; keep `requires-python = ">=3.11"`.
3. `release-pypi.yml` – the `crypto-licensing / python-slip39` workflow verbatim (tag trigger, environment: `pypi`, permissions: `id-token: write, pypa/gh-action-pypi-publish@release/v1`), extended to build and upload several projects from one repository.
4. Test the round trip on **TestPyPI** first. It is a separate pending publisher and costs nothing to get wrong.

Accept: `pip install alberta-buck-core` in a clean venv; from `buck_core.session` import `Web3Session` imports; the `py-v0.1.0` tag published it with no token anywhere.

9 Phase P4: PyPI – alberta-buck-kernel (binary wheels)

The only phase with real engineering in it.

- **abi3-py311 is already set** in both binding crates, which is the big simplification: **one wheel per platform**, not one per Python version, and it keeps working on Python 3.13, 3.14 and beyond with no rebuild.
- *Recommendation:* build with **maturin** rather than the current plain-cargo Makefile path. maturin knows abi3, emits correct wheel tags, handles the macOS `-undefined dynamic_lookup` flags that `core/rust/.cargo/config.toml` currently hand-rolls, and vendors workspace path dependencies into the sdist. Keep the existing `make core-build-py` for development – it is faster and already works.
- Target matrix (`cibuildwheel` or `maturin` in a matrix job): macOS arm64 + x86_64, manylinux_2_28 x86_64 + aarch64, Windows amd64. **Include linux-aarch64:** it is the gap that bites `pyrevm` today, and ARM servers are where a cheap game backend lives.
- Ship the dylib once (D5); verify the `ExtensionFileLoader` shim on every wheel in CI by importing all four modules and running the vector suite.
- The sdist must build standalone: a clean-room `pip install --no-binary :all: alberta-buck-kernel` in a container with only Rust installed.

Accept: wheels for all five targets on PyPI; `pip install alberta-buck-kernel` on a Linux ARM box gets a wheel, not a compiler; `BUCK_IDENTITY_BACKEND=kernel` `pytest` passes against the installed wheel with vectors byte-identical to the repo run.

10 Phase P5: PyPI – alberta-buck

Blocked on P3 (hard `buck_core` import), on P2.5 (artifacts, or the sim cannot deploy) and on D1 (readiness to publish the sim).

Work items:

1. **Dependency hygiene.** The current dependency list makes a research platform’s full weight mandatory for anyone who wants the library:

dependency	used by	move to
web3	everywhere	keep
python-dotenv	config	keep
py_ecc	wallet	keep
matplotlib	sim/plot_*.py, plotting tests only	[plots]
websockets	sim/server.py only	[server]
pyrevm	sim/pyrevm_backend.py only	[fast]

pyrevm especially: pinning it as a hard dependency means `pip install alberta-buck` **fails to build** on linux-aarch64 and on macOS with Python 3.13+, because upstream ships no wheels there. As an extra, it is opt-in and its absence degrades to the anvil backend, which is what the code already does.

2. **Add alberta-buck-core and alberta-buck-contracts to dependencies** (alberta-buck-kernel as the [kernel] extra), then **delete the `sys.path shim`** in `alberta_buck/__init__.py` – the code comment already anticipates this ("Removed once buck-core ships as a wheel").
3. **Align requires-python to ≥ 3.11** (alberta-buck-core requires it; abi3-py311 assumes it). Currently 3.10.
4. **Dynamic version** from `alberta_buck/version.py`, plus the tag-matches-version CI guard.
5. Decide what ships in the sdist. `package-data` is already carefully curated (Poseidon constants, golden renders, E2E fixtures, quote CSVs); confirm the test suite passes from an installed wheel, not just from the checkout.

Accept: `pip install alberta-buck[server,fast]` in a clean venv on Linux x86_64 launches `python -m alberta_buck.sim --backend pyrevm` successfully *outside any checkout*; `pip install alberta-buck` alone (no extras) imports and runs the non-plot tests on macOS ARM with Python 3.13.

11 Phase P6: what the game adds

The game is not one more package; it is a consumer that pulls the others into existence. Under D6/(3) its shape is:

board (projected) personal device shared state

-----		-----		-----
alberta-buck-game	<->	alberta-buck-game	<->	Holochain hApp
alberta-buck-core		alberta-buck-core		(zomes: Rust,
alberta-buck-kernel		alberta-buck-kernel		alberta-buck-identity,
(public: prices, pools,		(private: net worth,		alberta-buck-wallet
events, narrative)		holdings, decisions)		from crates.io)

New artifacts implied, none of them blocking the phases above:

- **alberta-buck-game** (npm) – rules, turn structure, the narrative price engine (trade wars, embargoes, a mill fire), the traditional-mortgage versus BUCK-issuance decision surface. The economics belong here; the monetary arithmetic stays in the kernel.
- **Holochain zomes** (Rust, in-repo, distributed as a `.happ` bundle rather than via a registry). They depend on `alberta-buck-identity` / `alberta-buck-wallet` from `crates.io` – the concrete reason P1 comes first. Note the standing caveat from the platform doc: the kernels are `std` today; the `no_std` flip is mechanical if a zome needs it.
- **The narrative price generator** is wanted by both the game (JS) and the research sim (`alberta_buck.sim.quotes`, Python). If it must agree across both, it belongs in the kernel with vectors; if it need not, keep two simple implementations and say so out loud.
- **Static hosting** for board and companion app – no registry involved; the existing `perry.kundert.ca` deck/demo pipeline is the model, and P2.5/(5) puts the contract artifacts there too.

Accept (as a deployment milestone, not a game milestone): a laptop with no Alberta Buck checkout runs `npm create ->` installs published packages `->` serves a board that a phone joins, with all monetary math executed by the published wasm kernel.

12 Phase P7: making the repository public

Independent of every phase above – publish first, open the repository when you choose. Checklist for when you do:

1. Re-run the secrets audit over full history (`.env` is clean today; re-verify, and check `foundry.lock`, `licensing.*`-style artifacts, RPC URLs embedded in committed JSON, and the `.claude/` directory).

2. Decide about the 216 MB of history. Rewriting it (dropping the big circom power tables and PDFs) is a one-time option that disappears the moment the repo is public and forked.
3. **README.org** as a front door: what this is, what installs, what the packages are, what the papers are.
4. **CONTRIBUTING**, **SECURITY.md** (a monetary system needs a disclosure address), issue templates.
5. Branch protection on **master**; require the **ci.yml** gate.
6. Only then do npm provenance attestations start being generated (P2).
7. The Solidity *sources* become public here; **alberta-buck-contracts** can then carry a source-verification link.

13 Risks and open questions

- **Publishing precedes the papers.** Source on PyPI is source in public. If the equilibrium findings or the rebalance policy are meant to land with a paper, sequence D1 accordingly – this is a strategic decision, not a technical one, and it is the single item on this page most likely to be regretted if made by default.
- **Irreversibility.** crates.io yanks but never deletes; PyPI never re-uses a filename; npm unpublish is 72 hours. Rehearse on TestPyPI, use `--dry-run`, and read `cargo package --list` before every first publish.
- **Unaudited contracts in a public registry.** Mitigated by prominent labelling, 0.x versions, and an empty `deployments.json` – but someone will eventually deploy them anyway. Consider whether the package should ship a constructor-level "testnet chain ids only" guard in the sim helpers rather than the contracts themselves.
- **CAL is an unfamiliar licence** (D2). Institutional counsel will read it rather than recognize it, and some will decline on unfamiliarity alone. Accepted deliberately – the friction is the instrument – but it makes the dual-licensing path (and therefore the CLA, and therefore your time answering licence questions) part of the operating cost, not an afterthought.

- **Binary wheel maintenance** is a standing tax: every new CPython, every new manylinux baseline. `abi3` removes the per-version half of it; the per-platform half remains. The mitigation is that `alberta-buck-kernel` is optional by construction (D5).
- **Version skew across three registries.** A JS game pinned to `alberta-buck-kernel@0.2` talking to a Python server on `alberta-buck-kernel==0.1` is a bit-identity violation that no test in this repository would catch. Mitigation: kernels version in lockstep (D4), and a published cross-registry vector suite that each language runs against its *installed* artifact rather than its checkout.
- **Contract/kernel drift.** The kernels are bit-identical to the contracts *at a commit*. Once both are published on independent version streams, nothing enforces the pairing. Mitigation: `compiler.json` records the commit; the kernel crates should record the contract version their vectors were generated from, and the vector suite should assert it.
- **npm supply chain.** Trusted publishing (no long-lived tokens) plus the `files` allowlist plus `npm publish --dry-run` in CI. No classic tokens, ever.
- **Holochain maturity** – unchanged from the platform doc’s assessment; the D6/(3) recommendation deliberately keeps the game’s monetary correctness independent of it.

14 Status board

Updated as phases complete; - is not started.

#	artifact / step	registry	version	status
-	D1 public-code boundary			-
-	D2 licence (CAL-1.0 kernels)			settled
-	D2 licence files written			DONE
-	D3 names (prefix rule)			settled
-	D4 version + tag scheme			settled
-	D5 pure/binary split			-
-	D6 game authority			-
0	P0 pre-flight (metadata, vectors, CI)			CI left
1	alberta-buck-math	crates.io	0.1.0	PUBLISHED
1	alberta-buck-identity	crates.io	0.1.0	PUBLISHED
1	alberta-buck-registry	crates.io	0.1.0	PUBLISHED
1	alberta-buck-wallet	crates.io	0.1.0	PUBLISHED
1	team ownership + trusted publishing	crates.io		-
2	alberta-buck-kernel	npm	0.1.1	PUBLISHED
2	alberta-buck-core	npm	0.1.1	PUBLISHED
2.5	alberta-buck-contracts	npm	0.1.0	-
2.5	alberta-buck-contracts	PyPI	0.1.0	-
3	alberta-buck-core	PyPI	0.1.0	-
4	alberta-buck-kernel	PyPI	0.1.0	-
5	alberta-buck	PyPI	0.1.0	-
6	alberta-buck-game + zomes	npm / -		-
7	repository public	GitHub		-